



Università degli Studi di Bologna
Facoltà di Ingegneria

ICCBD

UNIX files and
Single Primitive Atomicity

Antonio Corradi
Academic Year 2025/2026

UNIX: Files & Directories

UNIX has a very clean and neat idea to **simplify and make simple and understandable** its **semantics**

Only a few primitives on Files (and Directories) are atomic operations in the Kernel

Typical primitives

- **Operations on file** (*open/read/write/close – remove*)
- **Operations on directories** (*opendir/readdir/closedir – rmdir*)

The Kernel does

- grant **minimal atomicity** on single primitives
- not provide **any synchronization policy**

It offers only mechanisms

and does not support any policy among primitives

Atomicity

In UNIX, **atomicity** refers to the property of a **primitive** executed as a single, indivisible operation

it either completes entirely or not at all, with no intermediate state visible to other application processes or threads

Another Definition

A UNIX primitive is **atomic** if the kernel guarantees that its effects are applied as one **uninterruptible action**, so that no other process can observe a partial result

Clean UNIX semantics and very understandable

Atomicity in UNIX primitives ensures **kernel-level consistency**, not **application-level isolation**

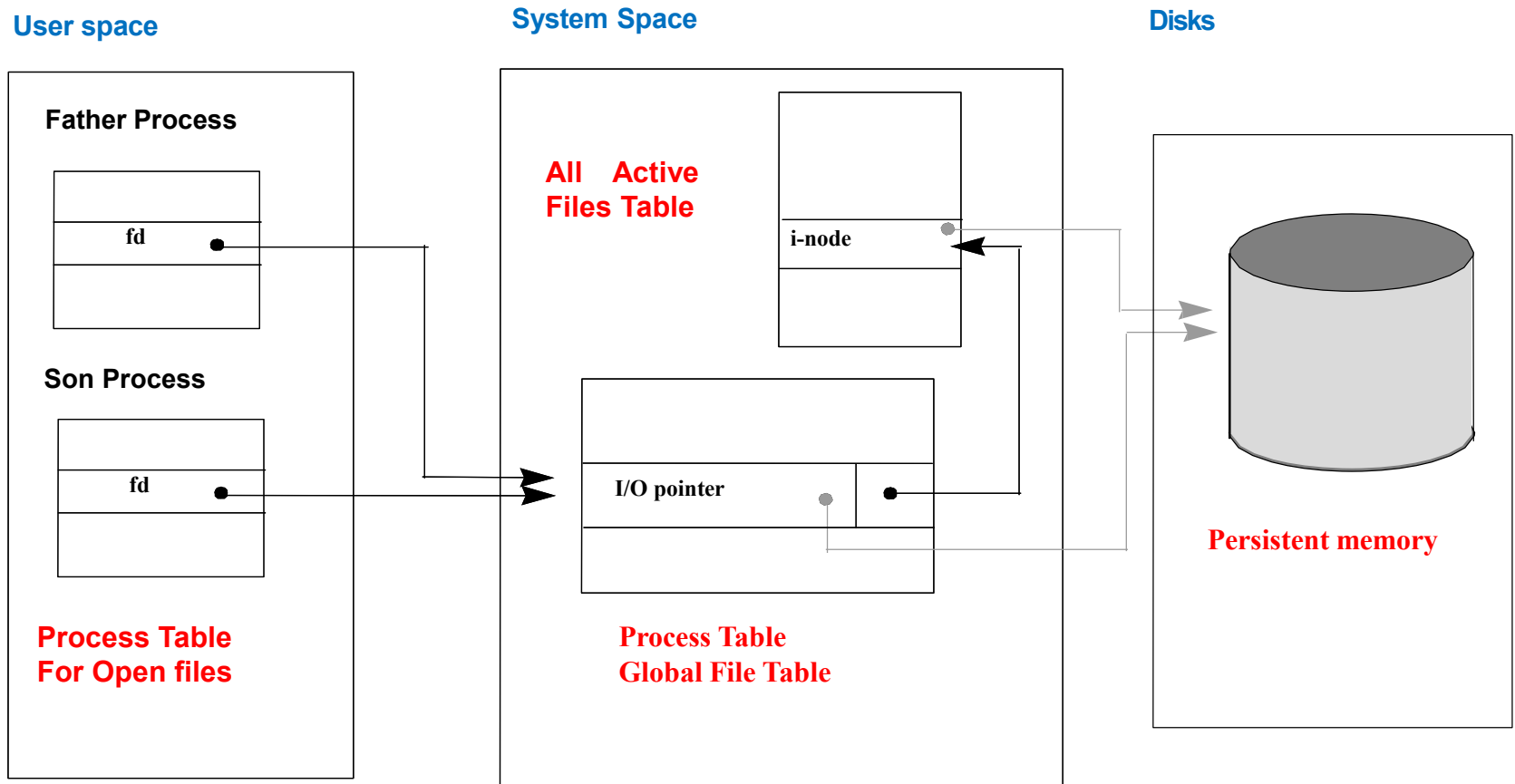
That is the base for predictable behavior under concurrency

Synchronization and consistency among primitives are left to the applications

Full picture

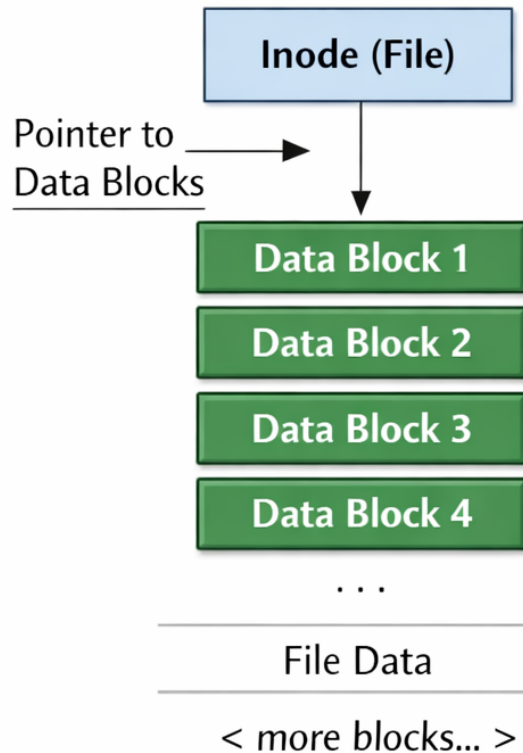
In **UNIX**, any **process may have** some open files (eventually shared in a process group)

The system keeps the connection to access real **blocks on disk** **via memory cache**

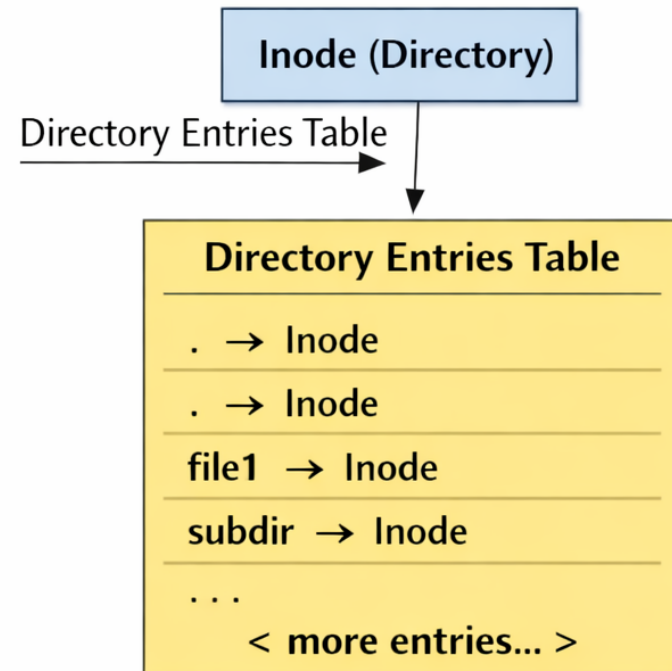


Files & Directories on Disk

Structure of a File in UNIX



Structure of a Directory in UNIX



Data Block



Directory Entry

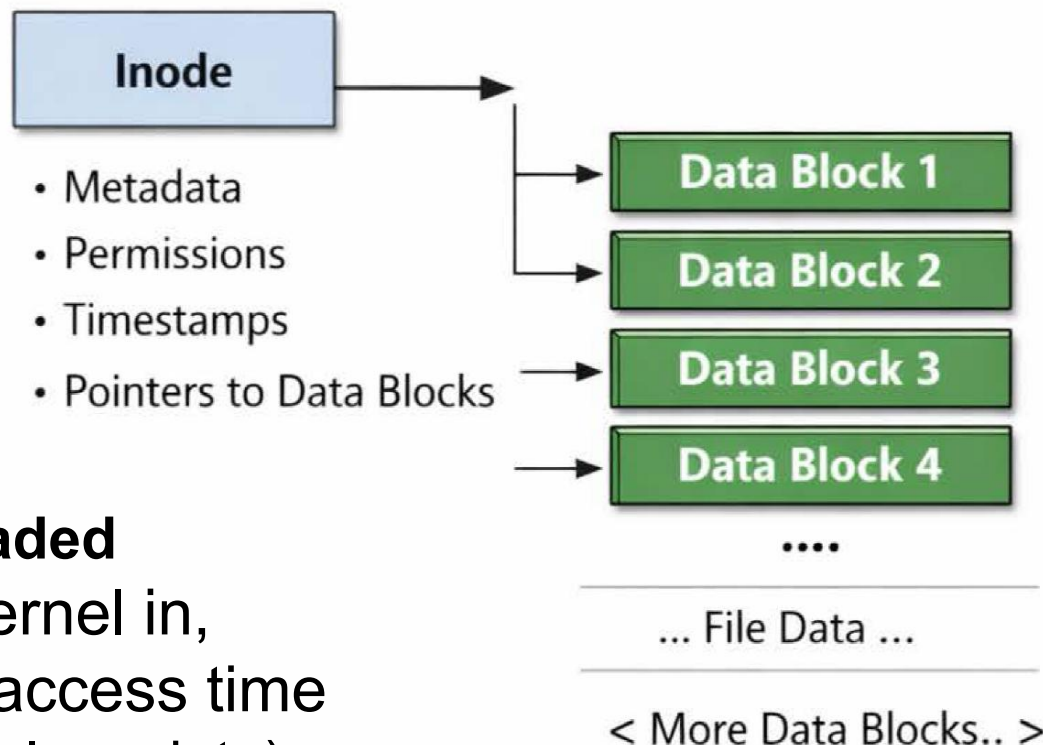


Inode

Files in memory and on Disk

In **UNIX**, a file is **managed and accessed** to by an **i-node** that is the **metalevel data structure** that keeps track of **all the blocks** constituting the file on the disk

File Structure in UNIX

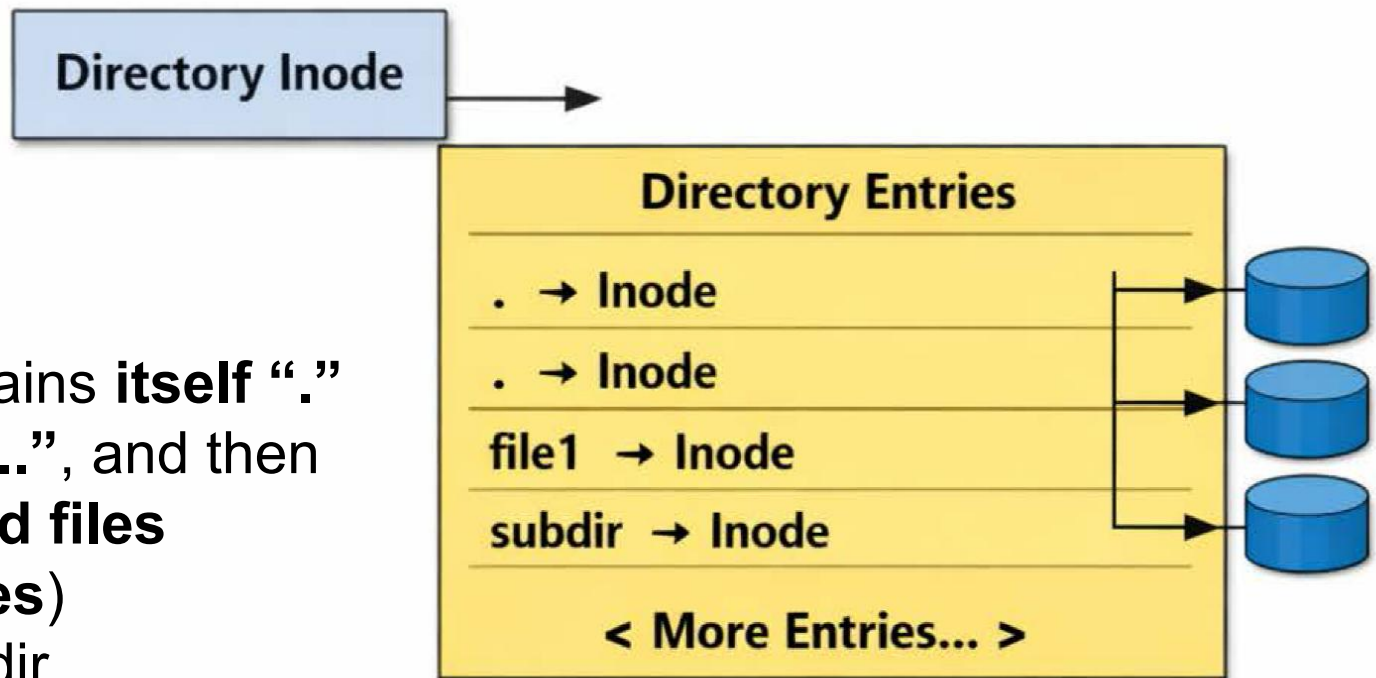


Block copies are loaded in memory by the kernel in, a cache to shorten access time (we make open, **r/w**, close data)

Directories on Disk

In **UNIX**, a **directory** is a special file (**specific ad-hoc primitives**) **to keep track of all files (and directories)** belonging to that metalevel structure **via i-nodes of them**

Directory Structure in UNIX



A directory contains **itself** “.” **and its father** “..”, and then all the **contained files** via (**their i-nodes**)
(we make opendir,
readdir, closedir)

Two processes write on the same file

In UNIX, 2 processes may have opened the same file with different I/O pointers (different opens in write and I/O pointers)

P1 write at position 0 “AAAAA”

P2 write at position 3 “BBBBB”

The operations are primitives (one is first, the other second) but **the ordering is not defined (non-determinism)**

Typically, the pattern in the file can be either one of the two

“AAABBBBB”

“AAAAABBB”

If the writes would insist in the same position

“AAAAA”

“BBBBB”

You cannot predict any final pattern that depends on the non-deterministic scheduling of primitives

Listing of a long directory

In UNIX you may **list recursively the list file from root**

You expect a snapshot, ... but the list make only separated primitives as if it is **many pictures to a moving target**

The kernel guarantees **each readdir() call is atomic**, but **not the whole traversal**

Since **the sequence of opendir() and readdir()** are not **atomic**, the recursive traversal print a **moving target**

- Some listed files are no longer there
- Other files are not listed

So, the entire list is **inconsistent**, and it is **not a snapshot**

A recursive directory listing under **concurrent modification** is **eventually consistent**, not an atomic snapshot

UNIX guarantees **atomicity per primitive only**, not across **sequences of primitives**